Overriding

CS300: Programming II

Cole Nelson

Learning Objectives

1. Be able to **use** `@Override` to override a method.
2. Be able to **use** `super.method()` to build on a parent's method.
3. Be able to **write** an `equals(Object)` method that uses `instanceof`.

Recap: `extends`

```
public class Textbook extends Book {
  private String courseCode;

  public Textbook(String title, String author, String courseCode) {
    super(title, author);
    this.courseCode = courseCode;
  }
}
```

A `Textbook` got `renew()`, `getTitle()`, and `toString()` from `Book` -- for free.

...but its `toString()` doesn't know about courses

```
System.out.println(calc);
```

```
Calculus, 9th Edition by James Stewart
 - Checked Out? true
 - Days Left: 30
 - Tags: []
```

Where's MATH 221? `calc` ran `Book` `'s` `toString()`.

Every class extends `Object`

Even `Book`! When you don't write `extends`, Java fills in `extends Object`.

`Object` gives every class a `toString()` and an `equals()`. That's where `Book@6d06d69c` came from, before `Book` wrote its own `toString()`.

Overriding

```
@Override
public String toString() {          // same name, parameters, and return type
  return "a Textbook!";             // replaces Book's version
}
```

Overriding replaces a method you inherited: same signature, new body.

Overriding vs. overloading

```
public String toString()                // override: same signature, replaces
public String toString(boolean full)    // overload: new parameters, adds
```

Overloading **adds** a method next to the old one.

Overriding **replaces** the old one.

`@Override` asks the compiler to check

```
@Override
public String tostring() {      // typo!
```

```
The method tostring() of type Textbook must override or implement
a supertype method
```

Without `@Override` this compiles -- and quietly adds a new method that nobody calls.

`super.toString()`

```java
@Override
public String toString() {
  return super.toString() + "\n - Course: " + this.courseCode;
}
```

Don't retype `Book`'s version -- **reuse it**. `super.` means *the version I inherited*.

Eclipse Exercise

Replace, don't extend

```
public class ReferenceBook extends Book {
  @Override
  public void renew() {
    throw new IllegalLoanException("Reference books stay in the library");
  }
}
```

No `super.renew()` here. A dictionary should **never** leave the library.

Eclipse Exercise

Get today's starter from Canvas.

TODOs 1a and 1b. Override `toString` in `Textbook`, reusing `Book`'s. Same for `Novel`, with its genre.

TODOs 2a and 2b. Override `renew()` in `ReferenceBook` so it throws. Be sure to override both `renew` methods.

Tophat Q1

A check on overriding and `super`.

→ Open Tophat to vote.

Two copies of Dune

```
Novel dune = new Novel("Dune", "Frank Herbert", "Science Fiction");
Novel dune2 = new Novel("Dune", "Frank Herbert", "Science Fiction");

System.out.println(dune == dune2);          // false: two distinct objects
System.out.println(dune.equals(dune2));     // false?!
```

`Object`'s `equals` just checks `==`. For books we want: same title, same author.

→ See the two objects in a visualization.

`equals(Object o)`

```java
@Override
public boolean equals(Object o) {
  if (o instanceof Book) {          // is o really a Book?
    Book other = (Book) o;          // then treat it as one
    return this.title.equals(other.title)
        && this.author.equals(other.author);
  }
  return false;                     // not a Book, so not equal
}
```

The parameter **must** be `Object` -- that's the method we're overriding.

`instanceof` , then a cast

```
if (o instanceof Book) {    // true for a Book, a Textbook, a Novel...
  Book other = (Book) o;    // the cast lets us use it as a Book
```

`other.title` works, too: `private` means *only code in `Book`*, not *only this object*.

Eclipse Exercise

TODO 3. Override `equals` in `Book`: the same title and author means equal. Now `dune.equals(dune2)` should print `true`.

Tophat Q2

A check on `equals`.

→ Open Tophat to vote.

Questions?